Aleksandr Sorokin
Sebastian Drumm
Lucas Chantelou
Jafar Alsukafi
Viggo Beck

Gihub Organisation

Indholdsfortegnelse

Intro.....2
Testing methodology.....2
Known bugs in code.....2
Black box tests.....3
First Name Black-Box Tests.....3
1. Equivalence Partitions.....3
2. Boundary Value Analysis.....3
3. Decision Table.....4
3. Justification for Representative Testing.....4
LastName Black-Box Tests.....5
1. Equivalence Partitions.....5
2. Decision Table.....5
BirthDate Black-Box Tests.....5
The rules are:.....5
1. Equivalence Partitions.....6
2. Boundary Value Analysis (3-Value).....7
3. Decision Table.....10
4. Justification for Representative Testing.....10
CPR (Black-box testing).....11
Test Design – CPR Generator.....11
1. Black-box Test Scenarios.....12
2. Equivalence Partitions.....12
2.1 Decision table full name birthday and gender.....13
2.2 Decision table for gender set last digits.....14
3. Boundary Value Analysis – CPR Length.....14
3.1 Boundary på random digit.....14
Code coverage.....14
Mobile phone number.....15
1. Equivalence Partitions.....15
2. Boundary Value Analysis.....16
3. Decision Table.....17
Justification for reduced test cases.....17
Statement Coverage.....17
Address.....18
Rules:.....18
Equivalence Partitions – Address.....18
Boundary value analysis.....23
Quality Gates.....29
PHPCODESNIFFER (Linter for PHP).....29

Errors and warnings found (image).....29
Autofix (image).....29
Errors and warnings (image).....30
Sonar Qube.....31
Image of code coverage using Sonar Qube.....31

Intro

This is a random person generator, it follows certain rules for creating a Danish person, example can be found here:
https://www.fakenamegenerator.com/

The goal of this assignment is to analyse the requirements and write tests for this system. We will go over black-box and white-box tests, end-to-end tests using PHPUnit, Postman and playwright. As there was no frontend server included in the frontend, we have added Node.js and use the serve package as this is static content.

We've implemented Continuous Integration with Github Actions and static code analysis with SonarQube.

Testing methodology

We're using Khorikov's methodology regarding the integration tests, meaning we will cover "happy paths", the longest ones that cover more integrations. He also states that for managed dependencies, we should use the actual dependencies.
Khorikov, V. (2020). *Unit Testing: Principles, Practices, and Patterns*. Manning.

Known bugs in code

We have found the following bugs in the code, we will not be correcting these but account for the failed tests by stating them here.

1. Gender rule in CPR (even for females, odd for males) isn't being enforced correctly
2. When attempting a POST request, the Content-Type header is set to text/html as opposed to application/json

It is optional to change the provided codebase, and we've decided not to.

Black box tests

First Name Black-Box Tests

1. Equivalence Partitions

Partition type	Description	Test Value	Expected result
Valid	Correct format	Peter A.	Valid
Invalid	Empty	“ “	Invalid
Invalid	Contains numbers	Peter1 A.	Invalid
Invalid	Contains special characters	Peter@ A.	Invalid
Invalid	Missing initial	Peter	Invalid

2. Boundary Value Analysis

Boundary	Description	Test value	Expected result
Below minimum	Just outside valid range	Peter	Invalid
Minimum valid	Smallest valid input	A B.	Valid
Typical valid	Normal, middle of range value	Peter A.	Valid

Just above invalid,	Name just outside valid range	Peter A	Invalid
Upper edge / Special case	Edge case lower case initial	Peter a.	Invalid

3. Decision Table

Case	First Name present?	Initial present?	Dot after initial?	Initial uppercase?	Letters only?	Result	Example
1	Yes	Yes	Yes	Yes	Yes	Valid	Peter A.
2	No	-	-	-	-	Invalid	“”
3	-	No	-	-	-	Invalid	Peter
4	-	-	No	-	-	Invalid	Peter A
5	-	-	-	No	-	Invalid	Peter a.
6	-	-	-	-	No	Invalid	Peter1 A.
7	-	-	-	-	No	Invalid	Peter@ A.

3. Justification for Representative Testing

- All valid first names are treated the same, so one typical example (e.g. *Peter A.*) is sufficient to represent the valid partition.
- Invalid cases are tested with a representative from each class: empty, missing initial, lowercase initial, numeric characters, and special characters.
- Boundary values are tested at the extremes of name length and initial correctness to ensure the system rejects names that are just outside valid limits.
- This approach ensures coverage of all functional rules for first names without needing to test every possible random name the generator can produce.

LastName Black-Box Tests

1. Equivalence Partitions

Partition type	Description	Test value	Expected value
Valid	Correct format	Jensen	Valid
Invalid	Empty	“”	Invalid
Invalid	Contains numbers	Jensen1	Invalid
Invalid	Contains special characters	Jensen@	Invalid

2. Decision Table

Case	Last Name present?	Letters only?	Result	Example
1	Yes	Yes	Valid	Jensen
2	No	-	Invalid	“”
3	Yes	No (numbers)	Invalid	Jensen1
4	Yes	No (symbols)	Invalid	Jensen@
5	Yes	No (space)	Invalid	Jensen Jens

BirthDate Black-Box Tests

The rules are:

Year: random between 1900 and current year (2026)

Month: random between 1 and 12

Day: depends on the month:

Months [1, 3, 5, 7, 8, 10, 12] → 1-31

Months [4, 6, 9, 11] → 1-30

Month 2 (February) → 1-28 (leap years ignored)

Output format: YYYY-MM-DD

1. Equivalence Partitions

Year

Partition Type	Partition	Test Value	Expected Result
Invalid	<0	-50	Invalid
Invalid	0	0	Invalid
Invalid	<1900	1850	Invalid
Valid	1900–2026	1983	Valid
Invalid	> 2026	2030	Invalid

Month

Partition Type	Partition	Test Value	Expected Result
Invalid	< 1	0	Invalid
Valid – 31-day months	{1, 3, 5, 7, 8, 10, 12}	7	Valid, day range 1-31
Valid – 30-day months	{4, 6, 9, 11}	6	Valid, day range 1-30
Valid – February	{2}	2	Valid, day range 1-28
Invalid	> 12	13	Invalid

Day (per month group)

Partition Type	Month Group	Day Partition	Test Value	Expected Result
Invalid	Any	< 1	0	Invalid
Valid	31-day month (e.g., 1)	1–31	15	Valid
Invalid	31-day month	> 31	32	Invalid
Valid	30-day month (e.g., 4)	1–30	15	Valid
Invalid	30-day month	> 30	31	Invalid
Valid	February (2)	1–28	14	Valid
Invalid	February	> 28	29	Invalid

Format

Partition Type	Description	Test Value	Expected Result
Valid	ISO 8601 format	"1983-07-15"	Valid
Invalid	Wrong separators	"1983/07/15"	Invalid
Invalid	Non-numeric content	"YYYY-MM-DD"	Invalid
Invalid	Empty string	""	Invalid

2. Boundary Value Analysis (3-Value)

Year Boundaries

Boundary	Test Value	Expected Result
Below min	1899	Invalid

Min (lower edge)	1900	Valid
Above min	1901	Valid
Mid-range	1963	Valid
Below max	2025	Valid
Max (upper edge)	2026 (Current)	Valid
Above max	2027	Invalid

Month Boundaries

Boundary	Test Value	Expected Result
Below min	0	Invalid
Min	1	Valid (31-day)
Above min	2	Valid (Feb)
Mid-range	6	Valid (30-day)
Below max	11	Valid (30-day)
Max	12	Valid (31-day)
Above max	13	Invalid

Day Boundaries – 31-day months (e.g., January)

Boundary	Test Value	Expected Result
Below min	0	Invalid
Min	1	Valid
Above min	2	Valid

Below max	30	Valid
Max	31	Valid
Above max	32	Invalid

Day Boundaries – 30-day months (e.g., April)

Boundary	Test Value	Expected Result
Below min	0	Invalid
Min	1	Valid
Above min	2	Valid
Below max	29	Valid
Max	30	Valid
Above max	31	Invalid

Day Boundaries – February

Boundary	Test Value	Expected Result
Below min	0	Invalid
Min	1	Valid
Above min	2	Valid
Below max	27	Valid
Max	28	Valid
Above max	29	Invalid

3. Decision Table

Case	Year valid? (1900-2026)	Month valid? (1-12)	Day valid for month?	Format YYYY-MM-DD?	Result	Example
1	Yes	Yes (31-day)	Yes (1-31)	Yes	Valid	1983-07-15
2	Yes	Yes (30-day)	Yes (1-30)	Yes	Valid	2001-04-22
3	Yes	Yes (Feb)	Yes (1-28)	Yes	Valid	1995-02-14
4	Yes	Yes (31-day)	No (32)	Yes	Invalid	1983-07-32
5	Yes	Yes (30-day)	No (31)	Yes	Invalid	2001-04-31
6	Yes	Yes (Feb)	No (29)	Yes	Invalid	1995-02-29
7	No (2030)	Yes	Yes	Yes	Invalid	2030-06-15
8	Yes	No (13)	—	Yes	Invalid	1983-13-15
9	Yes	Yes	Yes	No	Invalid	1983/07/15
10	No	No	No	No	Invalid	ABCD-XX-YY

4. Justification for Representative Testing

The birthdate generator has three independent numeric components (year, month, day) with the day range being conditionally dependent on the month. Testing one representative from each equivalence class plus boundary values at the edges of each range is sufficient because:

All years within 1900-2026 are treated identically by the generator

Months fall into exactly 3 functional groups (31-day, 30-day, February), and all months within a group behave identically

Days within a valid range for a given month group are treated identically

The deliberate omission of leap year handling means February is always capped at 28, which simplifies the partitioning (no need for a leap year sub-partition)

CPR (Black-box testing)

Test Design – CPR Generator

The CPR generator is tested using black-box testing techniques such as equivalence partitioning, decision tables, and boundary value analysis to ensure correct behavior under different input conditions and compliance with CPR rules.

The tests cover scenarios where required data (birth date, first name, last name, and gender) may be present or missing. The system automatically generates missing values, meaning all input combinations are valid but lead to different behaviors.

Equivalence partitioning is used to group inputs into categories like gender (female, male, or not set), presence of required data, and whether the final digit is even or odd. Representative values are selected to reduce the number of test cases.

Decision tables model combinations of conditions and expected outcomes, including handling missing data and rules for the final digit based on gender.

Boundary value analysis is applied to the CPR length and random digit. The expected length is 10 digits, but testing reveals that 11 digits can occur, indicating a defect. Boundary values (0 and 9) and near-boundary values (1 and 8) are also tested.

The `setCpr()` function is structured in a way that combines multiple responsibilities within a single method. It not only generates the CPR number but also ensures that required data such as birth date, name, and gender are set by calling other functions if they are missing.

This design makes the function highly dependent on internal state and side effects, as it conditionally initializes missing data before generating the CPR. As a result, the function is

harder to isolate and unit test using the traditional Arrange-Act-Assert (AAA) pattern, since there is little control over the setup (Arrange phase).

Instead, testing this function often requires integration-style tests, where the focus is on verifying the final output rather than testing individual components in isolation. The randomness introduced by `mt_rand()` and helper methods further reduces test predictability, making deterministic unit tests more difficult.

1. Test Scenarios

case	Situation	Expected
1	birthDate, firstName, lastName, gender are set	Existing values are used
2	birthDate is not set	birthDate is generated
3	firstName is not set	setFullNameAndGender()
4	lastName is not set	setFullNameAndGender()
5	gender is not set	setFullNameAndGender()
6	all values are missing	birthDate, full name, and gender are generated

2. Black box tests

2.2 Decision table full name, birthday and gender.

actions	Birthdate set?	firstName set?	lastName set?	gender sat?	Forventet handling
A1	True	True	True	True	Use existing data
A2	False	True	True	True	Generér birthDate

A3	True	False	True	True	generate full name and gender
A4	True	True	False	True	generate full name and gender
A5	True	True	True	False	generate full name and gender
A6	False	False	False	False	Generér birthDate, firstName, lastName, gender

Cases/ Rules	Length	Male	Female	Even	Odd	Date valid	Valid CPR
Valid male odd	T	T	F	F	T	T	T
Valid female even	T	F	T	T	F	T	T
Invalid gender	-	F	F	-	-	-	F
Invalid female odd	-	F	T	F	T	-	F
Invalid male even	-	T	F	T	F	-	F
Invalid length	F	-	-	-	-	-	F
Invalid date	-	-	-	-	-	F	F

2.3 Decision table for gender set last digits

Rules	Gender = female	Random last 4 digits is = even
R1	True	True
R2	True	False
R3	False	True
R4	False	False

2.4 Boundary Value Analysis – CPR Length

Partition type	Value	Expected
Invalid (below boundary)	0 - 9	Not possible / not generated
Valid (at boundary)	10	Valid CPR length
Invalid (above boundary)	11 - max	Bug case

Code coverage

After creating the tests and running code coverage for the first time on the tests covering setCpr(), I observed a relatively high coverage percentage. This indicates how central this function is within the system, suggesting that it represents a significant part of the business logic.

However, high code coverage does not necessarily mean that the tests are effective or that all defects are detected, as coverage only measures how much code is executed, not the quality of the tests.

The high coverage also suggests that the method depends on several other functions and internal state. This may indicate high coupling and low cohesion, which can negatively affect maintainability and testability.

To improve the design, the method could be refactored into smaller and more independent functions, making it easier to test in isolation and improving the overall code quality.

```
Classes: 33.33% (1/3)
Methods: 66.67% (14/21)
Lines:   81.76% (121/148)

DB
  Methods:  50.00% ( 1/ 2)   Lines:  70.00% (  7/ 10)
FakeInfo
  Methods:  64.71% (11/17)   Lines:  80.17% ( 97/121)
Town
  Methods: 100.00% ( 2/ 2)   Lines: 100.00% ( 15/ 15)
```

Mobile phone number

The mobile phone number in the system follows the Danish rules for beginnings and length. It is 8 numbers long and must be composed of integers with certain prefixes.

1. Black box tests - Equivalence Partitions

Partition	Description	Test case values	Expected Result
0	Length 0, empty number	“”	Invalid
< 8	Number length is too short	55555	Invalid
> 8	Number length is too long	999999999	Invalid
Valid – single-digit prefix	Number starts with 2 and is 8 numbers long	22345678	Valid
Valid – two-digit prefixes	Number starts with 30, 31, 40, 41, 42, 50, 51, 52, 53, 60, 61, 71, 81, 91, 92, 93	30123456	Valid
Valid – three-digit prefixes (ranges)	Number starts with 342, 344-349, 356-357, 359, 362, 365-366, 389, 398, 431, 441, 462, 466, 468, 472, 474, 476, 478, 485-486, 488-489, 493-496, 498-499, 542-543,	34234567	Valid

	545, 551-552, 556, 571-574, 577, 579, 584, 586-587, 589, 597-598, 627, 629, 641, 649, 658, 662-665, 667, 692-694, 697, 771-772, 782-783, 785-786, 788-789, 826-827, 829		
Invalid prefix	Number starts with 1, 32, 360	36000000	Invalid
Invalid – non-numeric characters	Contains letters/symbols	2A345678	Invalid

2. Boundary Value Analysis

Boundary	Description	Test Value	Expected Result
Lower digit boundary (7)	Minimum length	2000000 (7 digits)	Invalid
Nominal valid (8)	Middle valid number	20012345	Valid
Upper digit boundary(9)	Maximum length	200000000 (9 digits)	Invalid
Prefix just below valid prefix (342)	Number just outside valid prefix	34199999	Invalid
Prefix lower edge valid prefix (342)	First number in valid range	34200000	Valid
Prefix just above lower edge valid prefix(342)	Just above first number in valid range	34200001	Valid
Prefix middle value (342)	Middle valid number	34255555	Valid
Prefix just below upper edge (342)	Just below last number in valid range	34299998	Valid

Prefix upper edge (342)	Last number in valid range	34299999	Valid
Prefix just above (342)	Number just outside valid prefix	34300000	Invalid

3. Decision Table

Case	8 digits?	Numeric only?	Pre-fix valid?	Result	Example
1	Yes	Yes	Yes	Valid	34212345
2	No	-	-	Invalid	3421234
3	-	No	-	Invalid	342A1245
4	-	-	No	Invalid	15123456

Justification for reduced test cases

The phone number is randomly generated, so we cannot reliably test for boundaries, this means that our tests only account for length, if it’s numeric and if the generated number falls within the allowed prefixes.

Whitebox tests - Decision Coverage

All methods that return a phone number, do so with the object's private $phone string.
This is set in the constructor with setPhone() and no other places. The setPhone() method doesn’t have any branching, however there is a loop which loops 5-7 times depending on the length of the prefix. Since there isn’t and won’t be a prefix of 8 length, this loop will always run. Meaning all decisions always get tested as long as we retrieve the phone number in any of the ways provided.

This test has 100% decision and statement coverage of the phone for example:

```
public function testPhoneLength(): void
{
    $phone = $this->fakeInfo->getPhoneNumber();
    $this->assertEquals(8, strlen($phone), "Phone number length should be 8");
}
```

Address

Rules:

Street:
- Must contain only alphabetic characters
- Must not be empty
- No digits or special characters are allowed

Number:
- Must be a number between 1-999
- May optionally be followed by a single uppercase letter. (fx. 43b)
- Lowercase letters or symbols are not allowed

Floor:
- Must be either:
    - “St” (ground floor), or
    - A number between 1-99
- Any other values are invalid

Door:
The door field supports multiple formats:
- One of the following fixed values: “th”, “mf”, “tv”
- A number between 1-50
- A lowercase letter optionally followed by a dash, and then followed by 1 to 3 digits for example: “c3” & “d-14”

Invalid cases include:
- Uppercase letters
- Missing numeric part
- More than 3 digits

Postal code and town
- Must a be a valid combination from the provided SQL dataset
- Postal code must be exactly 4 digits
- The town must match the corresponding postal code

Examples of valid combinations is:
- 2000 - Frederiksberg
- 8000 - Aarhus C

Invalid cases:
- Non-existing postal code
- Mismatch between postal code and town
- Incorrect format

Equivalence Partitions – Address

Street:

Partition Type	Partition	Test Value	Expected Result
Invalid	Empty string	“”	Invalid
Invalid	Contains digits	“Main1”	Invalid
Invalid	Contains special characters	“Road@”	Invalid
Valid	Alphabetic characters only	“Birkevej”	Valid
Valid	Multiple alphabetic characters	“Nørrebrogade”	Valid

Number:

Partition Type	Partition	Test Value	Expected Result

Invalid	Less than 1	0	Invalid
Valid	Number in range 1–999	43	Valid
Valid	Number in range 1–999 with uppercase letter	43B	Valid
Invalid	Greater than 999	1000	Invalid
Invalid	Number with lowercase suffix	43b	Invalid
Invalid	Number with symbol suffix	43-	Invalid

Floor:

Partition Type	Partition	Test Value	Expected Result
Valid	Ground floor	“st”	Valid

Valid	Number in range 1–99	4	Valid
Invalid	Less than 1	0	Invalid
Invalid	Greater than 99	100	Invalid
Invalid	Other text values	“ground”	Invalid
Invalid	Uppercase variant	“ST”	Invalid

Door:

Partition Type	Partition	Test Value	Expected Result
Valid	Fixed value th	“th”	Valid
Valid	Fixed value mf	“mf”	Valid
Valid	Fixed value tv	“tv”	Valid

Valid	Number in range 1–50	12	Valid
Valid	Lowercase letter + digits	“c3”	Valid
Valid	Lowercase letter + dash + digits	“d-14”	Valid
Invalid	Number below range	0	Invalid
Invalid	Number above range	51	Invalid
Invalid	Uppercase letter pattern	“D-14”	Invalid
Invalid	Missing numeric part	“c”	Invalid
Invalid	Too many digits	“c1234”	Invalid

Postal code and town:

Partition Type	**Partition**	**Test Value**	**Expected Result**

Valid	Existing postal code and matching town	2000 / Frederiksberg	Valid
Valid	Existing postal code and matching town	8000 / Aarhus C	Valid
Invalid	Existing postal code with wrong town	2000 / Aarhus C	Invalid
Invalid	Non-existing postal code	9998 / Unknown	Invalid
Invalid	Postal code wrong length	200 / Frederiksberg	Invalid

Boundary value analysis

Number

Boundary	Test Value	Expected Result
Below minimum	0	Invalid

Minimum (Boundary)	1	Valid
Above minimum	2	Valid
Mid-range	500	Valid
Below maximum	998	Valid
Maximum (Boundary)	999	Valid
Above maximum	1000	Invalid

Floor:

Boundary	Test Value	Expected Result
Below minimum	0	Invalid

Minimum (Boundary)	1	Valid
Above minimum	2	Valid
Mid-range	50	Valid
Below maximum	98	Valid
Maximum (Boundary)	99	Valid
Above maximum	100	Invalid

Door (numeric)

Boundary	Test Value	Expected Result
Below minimum	0	Invalid
Minimum (Boundary)	1	Valid

Above minimum	2	Valid
Mid-range	25	Valid
Below maximum	49	Valid
Maximum (Boundary)	50	Valid
Above maximum	51	Invalid

Door (letters + digits)

Boundary	Test Value	Expected Result
Minimum digits	“a1”	Valid
Mid valid	“b12”	Valid
Maximum digits	“c999”	Valid

Above maximum digits	“d1000”	Invalid

Decision table - Address

Case	Street alphabetic?	Number valid?	Floor valid?	Door valid?	Postal code/town pair valid?	Result	Example
1	Yes	Yes	Yes	Yes	Yes	**Valid**	Birkevej, 43B, 2, th, 2000 Frederiksberg
2	**No**	Yes	Yes	Yes	Yes	**Invalid**	Birkevej1, 43B, 2, th, 2000 Frederiksberg
3	Yes	**No**	Yes	Yes	Yes	**Invalid**	Birkevej, 1000, 2, th, 2000 Frederiksberg
4	Yes	Yes	**No**	Yes	Yes	**Invalid**	Birkevej, 43B, 100, th, 2000 Frederiksberg

5	Yes	Yes	Yes	**No**	Yes	**Invalid**	Birkevej, 43B, 2, 51, 2000 Frederiksberg
6	Yes	Yes	Yes	Yes	**No**	**Invalid**	Birkevej, 43B, 2, th, 2000 Aarhus C
7	Yes	Yes	Yes	Yes	Yes	**Valid**	Nørregade, 12, st, d-14, 8000 Aarhus C
8	Yes	Yes	Yes	Yes	**No**	**Invalid**	Nørregade, 12, st, d-14, 9998 Unknown

Quality Gates

PHPCODESNIFFER (Linter for PHP)

Errors and warnings found (image)

```
lucaslevy@MacBook-Pro-tilhrende-Lucas-2 fake_info_backend % phpcs --standard=PSR12 --extensions=php src/ index.php

FILE: /Users/lucaslevy/VSCodeProjects/fake_info_backend/src/Town.php
----------------------------------------------------------------------------------------------------------------------------------
FOUND 1 ERROR AND 1 WARNING AFFECTING 2 LINES
----------------------------------------------------------------------------------------------------------------------------------
  1 | WARNING | A file should declare new symbols (classes, functions, constants, etc.) and cause no other side effects, or it should execute logic with side effects, but should not do both. The
    |         | first symbol is defined on line 12 and the first side effect is on line 3.
 12 | ERROR   | Each class must be in a namespace of at least one level (a top-level vendor name)
----------------------------------------------------------------------------------------------------------------------------------

FILE: /Users/lucaslevy/VSCodeProjects/fake_info_backend/src/FakeInfo.php
----------------------------------------------------------------------------------------------------------------------------------
FOUND 1 ERROR AND 2 WARNINGS AFFECTING 3 LINES
----------------------------------------------------------------------------------------------------------------------------------
  11 | ERROR   | Each class must be in a namespace of at least one level (a top-level vendor name)
 267 | WARNING | Line exceeds 120 characters; contains 133 characters
 303 | WARNING | Line exceeds 120 characters; contains 155 characters
----------------------------------------------------------------------------------------------------------------------------------

FILE: /Users/lucaslevy/VSCodeProjects/fake_info_backend/src/DB.php
----------------------------------------------------------------------------------------------------------------------------------
FOUND 13 ERRORS AND 1 WARNING AFFECTING 9 LINES
----------------------------------------------------------------------------------------------------------------------------------
  1 | WARNING | [ ] A file should declare new symbols (classes, functions, constants, etc.) and cause no other side effects, or it should execute logic with side effects, but should not do both.
    |         |     The first symbol is defined on line 11 and the first side effect is on line 9.
  1 | ERROR   | [x] Header blocks must be separated by a single blank line
  1 | ERROR   | [x] End of line character is invalid; expected "\n" but found "\r\n"
  3 | ERROR   | [x] Whitespace found at end of line
  4 | ERROR   | [x] Whitespace found at end of line
  8 | ERROR   | [x] Header blocks must be separated by a single blank line
 11 | ERROR   | [ ] Each class must be in a namespace of at least one level (a top-level vendor name)
 11 | ERROR   | [x] Opening brace of a class must be on the line after the definition
 11 | ERROR   | [x] Whitespace found at end of line
 17 | ERROR   | [x] Opening brace should be on a new line
 25 | ERROR   | [x] Whitespace found at end of line
 36 | ERROR   | [x] Opening brace should be on a new line
 40 | ERROR   | [x] Expected 1 newline at end of file; 0 found
 40 | ERROR   | [x] A closing tag is not permitted at the end of a PHP file
----------------------------------------------------------------------------------------------------------------------------------
PHPCBF CAN FIX THE 12 MARKED SNIFF VIOLATIONS AUTOMATICALLY
----------------------------------------------------------------------------------------------------------------------------------

FILE: /Users/lucaslevy/VSCodeProjects/fake_info_backend/index.php
----------------------------------------------------------------------------------------------------------------------------------
FOUND 0 ERRORS AND 1 WARNING AFFECTING 1 LINE
----------------------------------------------------------------------------------------------------------------------------------
  1 | WARNING | A file should declare new symbols (classes, functions, constants, etc.) and cause no other side effects, or it should execute logic with side effects, but should not do both. The
    |         | first symbol is defined on line 5 and the first side effect is on line 3.
----------------------------------------------------------------------------------------------------------------------------------

Time: 64ms; Memory: 10MB
lucaslevy@MacBook-Pro-tilhrende-Lucas-2 fake_info_backend %
```

Autofix (image)

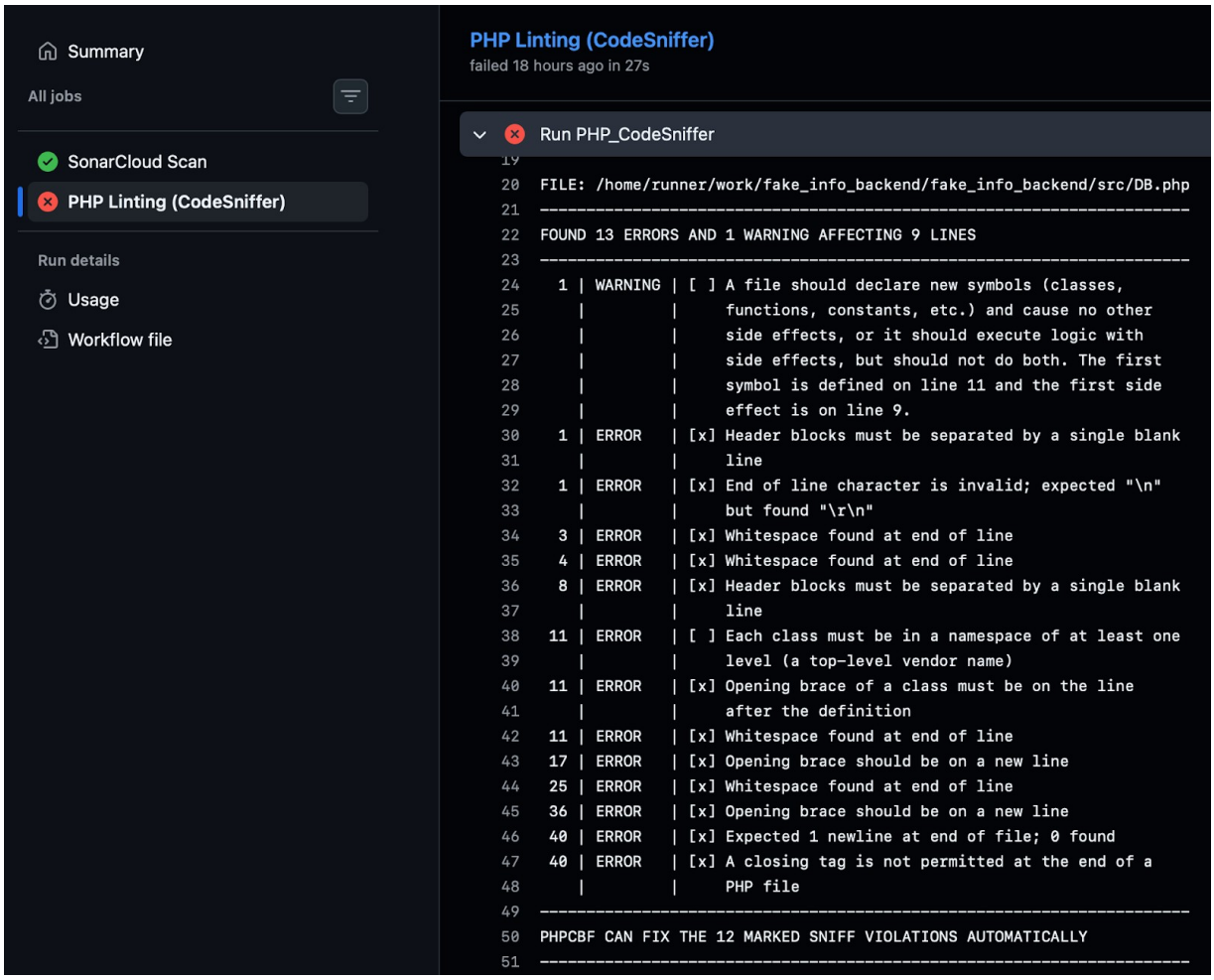
```
lucaslevy@MacBook-Pro-tilhrende-Lucas-2 fake_info_backend % phpcbf --standard=PSR12 --extensions=php src/ index.php

PHPCBF RESULT SUMMARY
----------------------------------------------------------------------------------------
FILE                                                                  FIXED  REMAINING
----------------------------------------------------------------------------------------
/Users/lucaslevy/VSCodeProjects/fake_info_backend/src/Town.php        8      2
/Users/lucaslevy/VSCodeProjects/fake_info_backend/src/FakeInfo.php    55     3
/Users/lucaslevy/VSCodeProjects/fake_info_backend/src/DB.php          FAILED TO FIX
/Users/lucaslevy/VSCodeProjects/fake_info_backend/index.php           4      1
----------------------------------------------------------------------------------------
A TOTAL OF 56 ERRORS WERE FIXED IN 4 FILES
----------------------------------------------------------------------------------------
PHPCBF FAILED TO FIX 1 FILE
----------------------------------------------------------------------------------------

Time: 246ms; Memory: 12MB
lucaslevy@MacBook-Pro-tilhrende-Lucas-2 fake_info_backend %
```

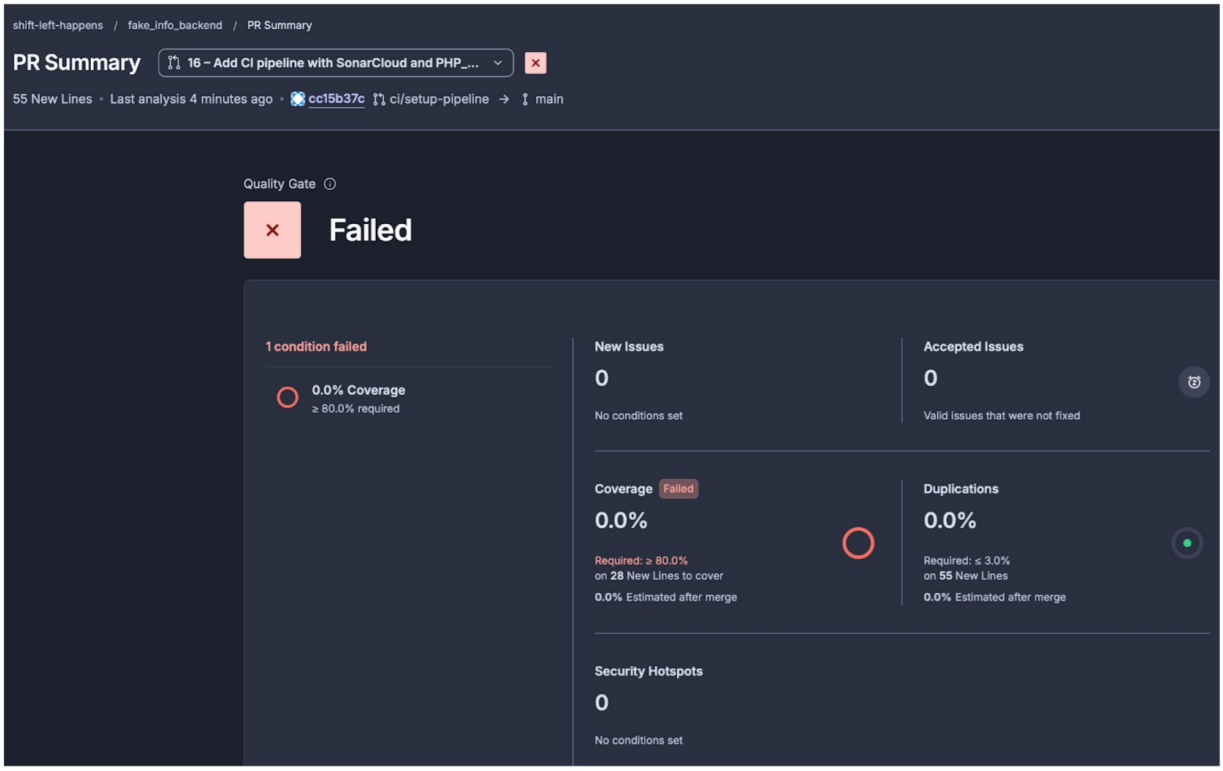
PHP CodeSniffer was used with the PSR-12 standard to analyse code quality and style. The tool identified multiple issues related to formatting, structure, and coding conventions, such as missing namespaces, long lines, and inconsistent brace placement. Using PHPCBF, 56 issues were automatically fixed, improving code readability and maintainability. Some issues required manual correction, showing that automated tools cannot resolve all problems.

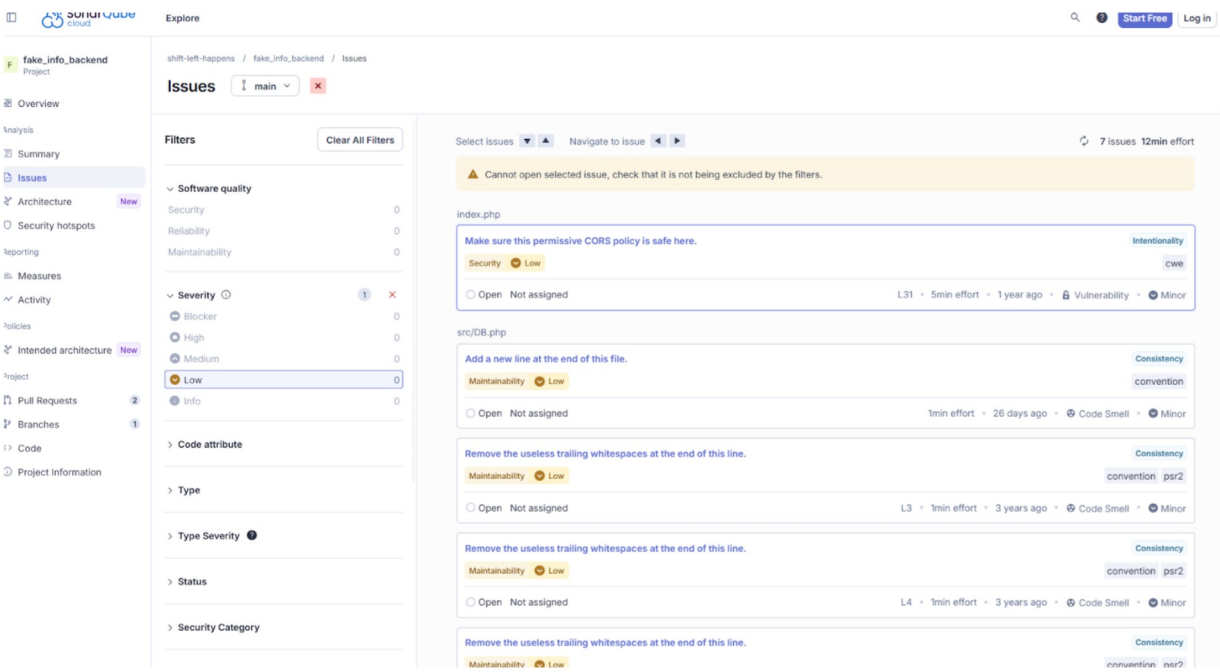
Errors and warnings (image)



Sonar Qube

Image of code coverage using Sonar Qube





SonarQube was used as a static analysis tool to identify issues in the code without executing the application. The analysis detected 7 issues with an estimated remediation effort of 12 minutes. Most of the findings were low-severity maintainability issues, such as trailing whitespaces and missing newline at end of file, but one security-related issue was also reported regarding a permissive CORS policy. This indicates that static analysis is useful for detecting both code quality problems and potential security concerns early in development.